ALPHA-DECAY: Autonomous Financial Forensic Research Agent

System Architecture, Multi-Agent Nodes, Deterministic Math Sandbox & Development Timeline

ATMOS 2026 // BITS PILANI HYDERABAD
Track C: Consumer, FMCG & Retail (NIFTY 50)

REPOSITORY: github.com/aryanmsrh/alphadecay

1. Executive Overview: Autonomous Discovery vs. Summarisation

Traditional financial AI tools act as superficial news summarizers or chat wrappers that hallucinate arithmetic and exhibit confirmatory bias. ALPHA-DECAY is an autonomous financial research agent built for the ATMOS 2026 Hackathon ('Finding the Next Signal'). Given any NIFTY 50 FMCG company from Track C, ALPHA-DECAY independently ingests public statutory filings, derives non-obvious mathematical accounting relationships, forms an original hypothesis, subjects it to adversarial falsification challenges, and generates an auditable, evidence-backed research deliverable meeting all 8 brief requirements.

2. Autonomous Investigation Process Workflow (6 Phases)

Phase	Engine Component	Operational Execution & Forensic Safeguards
Phase 1: Ingestion	Statutory Scrapers (Screener & BSE)	Harvests multi-year quarterly P&L;, annual balance sheets, and cash flow statements from audited SEBI LODR filings; extracts verbatim dialogue blocks from official BSE concall PDFs via PyPDF.
Phase 2: Math Sandbox	Deterministic Python Engine (analytics.py)	100% pure Python execution. Enforces strict period matching: annual CFO benchmarked against audited annual EBITDA ex-Other Income; quarterly sales YoY matched to identical quarter t-4. Zero LLM math.
Phase 3: Triage	Forensic Scoring & Ranking (forensics.py)	Ranks observed divergences into candidate anomalies by severity (/10): CFO-to-EBITDA decoupling, supplier payable stretches, DSO debtor velocity shifts, inventory DIO expansions, and margin wedges.
Phase 4: Adversarial Loop	Cyclic Pipeline (cyclic_pipeline.py)	Investigative Agent formulates a causal hypothesis. Disproving Agent acts as skeptical defense, testing whether ordinary FMCG commercial dynamics explain the shift. If broken down, Attempt 2 is refined.
Phase 5: Adjudication	Binary Verification Gate	Emits strictly binary verdicts: HYPOTHESIS PASSED (withstood adversarial scrutiny) or HYPOTHESIS FAILED (refuted by operational context). Synthesizes the official 8-point research brief.
Phase 6: Delivery	ReportLab PDF & Flask Quant Terminal	Renders a publication-grade 2-page dark research report, dual-axis trajectory chart, and interactive web dashboard featuring live execution polling, audited formulas, and direct BSE clickable links.

3. Multi-Agent Architecture: The Nodes & Cyclic Adversarial Logic

ALPHA-DECAY utilizes a specialized multi-agent graph with distinct epistemic boundaries. To eliminate confirmation bias, no single agent is allowed to declare a finding without adversarial cross-examination:

Agent / Node	Role & Epistemic Mandate	Implementation Mechanics & Failure Modes Prevented
Investigative Agent (Prosecution)	Formulates causal hypotheses connecting mathematical divergences to operational anomalies.	Analyzes top-ranked anomaly candidates against concall transcripts. Pinpoints non-obvious operational explanations (e.g., vendor payable optimization, marketing accrual cash drain).
Disproving Agent (Defense)	Adversarial stress-tester. Seeks to refute the hypothesis using benign commercial context.	Challenges hypotheses using structural FMCG realities (e.g. negative CCC of -89 days proving supplier leverage, raw material price buffering). Breaks down ungrounded claims.
Cyclic Refinement (Attempt 1 -> 2)	Self-correcting iterative retry loop across anomalous candidates.	If Attempt 1 is refuted by the defense, the agent synthesizes defense critique to formulate an airtight Attempt 2, or abandons the thesis and marks it as refuted.
Adjudication Node (Independent Judge)	Enforces strictly binary terminal verdicts and audits evidence strength.	Evaluates whether the defense broke down the hypothesis. Assigns binary HYPOTHESIS PASSED or HYPOTHESIS FAILED, audits evidence trail, and frames falsification criteria.

4. Deterministic Financial Math Sandbox: Exact Accounting Identities

To guarantee mathematical integrity, arithmetic is decoupled from LLM inference. All indicators are calculated directly in pure Python using audited balance sheets and SEBI LODR disclosures:

Forensic Indicator	Exact Formula	Period Alignment	Forensic Significance & Diagnostic Insight
CFO-to-EBITDA Ratio	`(Audited_CFO / Audited_Operating_Profit) * 100`	Audited Annual to Annual	Earnings quality test. A drop below 80% (e.g. HUL: 80.9% to 73.1%) reveals operating profit decoupling from physical cash flow.
EBITDA (ex-Other Inc)	`Net_Sales - Total_Operating_Expenses`	Quarterly & Annual	Isolates pure manufacturing/distribution operating profit, stripping non-operating treasury income gains.
Cash Conversion Cycle	`DSO (Debtor) + DIO (Inv) - DPO (Payable)`	Audited Annual	Measures working capital financing. A deeply negative cycle (-89d for HUL) proves vendors are funding operational inventory.
Debtor / DSO Velocity	`(Trade_Receivables / Audited_Sales) * 365`	Annual Audited	Distributor credit terms. Measures how fast wholesale trade partners settle receivables (HUL: improved 23d -> 19d).

ALPHA-DECAY // Technology Stack, Development Timeline & Deliverable Compliance

REPOSITORY: github.com/aryanmsrh/alphadecay

5. Verified Technology Stack & Production Infrastructure (Audited Against Codebase)

Layer	Actual Technology Used	Codebase Verification & Architectural Implementation Details
Core & Runtime	Python 3.10+, Flask 3.1, Atomic JSON Store	Pure Python deterministic core; lightweight Flask microservice (run_web.py); transactional JSON run store (webapp/run_store.py) with atomic .tmp swap.
LLM & Reasoning	Google Gemini 3.5 Flash Lite + Ollama (DeepSeek-R1)	Direct HTTP client (httpx) with high thinking level; cloud Gemini 3.5 Flash Lite primary; local Ollama deepseek-r1:7b offline reasoning fallback.
Ingestion & Harvest	Requests, BeautifulSoup4, PyPDF, Tavily API	Screener.in scraping via requests/BS4; official BSE concall PDFs downloaded via httpx and parsed via PyPDF; targeted financial search via Tavily.
Exporters & Visuals	ReportLab 5.0, Matplotlib 3.11, Chart.js	ReportLab custom flowables for strictly 2-page dark research PDF; Matplotlib dual-axis sales vs margin chart; Chart.js in browser workstation.
Frontend UI	Pitch-Black Quant Terminal (#070709)	Space Grotesk + Plus Jakarta Sans + JetBrains Mono typography; zero neon glow; responsive math cards; micro-elevation shadows.

6. Development Timeline: Day-by-Day Engineering Milestones (16th – 19th Oct 2026)

Date & Phase	Milestone Target	Key Technical Hurdles Solved & Architecture Built
16 October 2026 Day 1: Foundations	Problem Framing & Data Ingestion Scaffolding	Scoped Track C (Consumer, FMCG & Retail). Built robust Screener.in scraper parsing audited quarterly P&L;, annual balance sheets, and cash flows. Engineered BSE India concall PDF downloader and dialogue extractor.
17 October 2026 Day 2: Deterministic Math	Pure Python Analytics & Period Alignment	Eliminated LLM arithmetic hallucinations by writing pure Python formulas for YoY revenue growth, EBITDA ex-other income, CFO conversion, DIO, DSO, and CCC. Resolved quarterly-vs-annual mismatch by locking audited annual period baselines.
18 October 2026 Day 3: Adversarial Pipeline	Cyclic Agent Loop & ReportLab Exporter	Architected the 2-attempt adversarial challenge loop (Investigative Agent vs Disproving Agent). Built binary adjudication verdict logic (HYPOTHESIS PASSED / FAILED). Authored custom ReportLab canvas strictly constrained to 2 pages.
19 October 2026 Day 4: Polish & Packaging	Quant Terminal UI, Bug Fixes & Live Auditing	Redesigned UI to pitch-black quant workstation (#070709) with Space Grotesk / JetBrains Mono typography. Fixed double-sign (+- / ++) artifacts and hardcoded significance strings. Tested clean run on HUL with verified 2-page PDF and merged to main.

7. Official 8-Point Hackathon Deliverable Brief Compliance

Brief Requirement	AlphaDecay Implementation	Brief Requirement	AlphaDecay Implementation
1. The Discovery	One clear sentence describing the non-obvious finding.	5. Numbers	Deterministic calculations & formulas substantiating finding.
2. Why It Matters	3 to 5 bullets detailing operational & capital significance.	6. Counterargument	Strongest alternative explanation evaluated by defense agent.
3. Evidence	Multiple independent sources with direct, clickable links.	7. Falsification	Explicit future empirical condition that would disprove finding.
4. Timeline	Chronological emergence across sequential reporting periods.	8. Evidence Strength	Strong, Moderate, or Weak rating with audit rationale.

8. Key Architectural Edge for Finals Live Testing

1. Zero Arithmetic Hallucination: Pure Python deterministic calculations guarantee accurate numbers during live evaluations.
2. Multi-Company Generalizability: Works out-of-the-box across any Track C listed company (HUL, ITC, Nestle, Britannia, Tata Consumer).
3. Dual-Provider Resilience: Seamless switch to local Ollama (deepseek-r1:7b) via --offline routes LLM reasoning locally without cloud API keys.
4. Complete Auditability: Every claim links back to SEBI LODR filings, eliminating subjective LLM assertions.